

rwx

Security by File Permissions

Controlling who can read, write, and execute your files

```
$ ls -l notes.txt
```

What We'll Cover

01

Permission model

Owner, group, others — and read/write/execute

02

Octal notation

Representing permissions as numbers (e.g. 755)

03

chmod

Changing permissions — symbolic and numeric modes

04

chown / chgrp

Changing who owns a file, and its group

05

umask

The default permission mask for new files

Why File Permissions Matter

Linux is a multi-user system — permissions decide who can touch what

- Many users share one machine — permissions stop one user from reading or deleting another's files
- System files (passwords, kernel configs) must be protected from accidental or malicious changes
- Every file and directory has exactly one owner, one group, and a set of rules for everyone else
- Three permissions (read, write, execute) × three identities (owner, group, others) = the entire model

Owner (u)

The single user who created / owns the file

Group (g)

A set of users sharing access, e.g. "students"

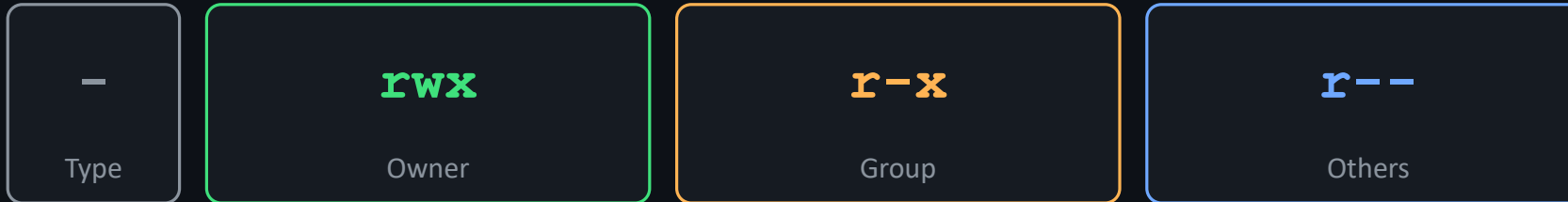
Others (o)

Everyone else on the system

Reading Permissions: ls -l

Every file listing encodes its type and permissions in the first 10 characters

```
$ ls -l notes.txt
-rwxr-xr-- 1 student students 1024 Sep 3 14:20 notes.txt
```



First character = file type: `-` regular file, `d` directory, `l` symbolic link

Remaining 9 characters = three groups of `rwx` for owner, group, others (in that fixed order). A dash `-` means that permission is denied.

In this example: owner (student) can read, write, execute; group (students) can read and execute but not write; everyone else can only read.

What r, w, x Actually Mean

The same letter means something different for a file versus a directory

Permission	On a FILE	On a DIRECTORY
r (read)	View the file's contents (e.g. cat)	List the directory's contents (e.g. ls)
w (write)	Modify or overwrite the file's contents	Create, delete, or rename entries inside it
x (execute)	Run the file as a program or script	Enter the directory (cd) and access items inside
Common trap: a directory without execute (x) can't be entered even if it has read (r) — you'll see names but can't cd into it or open anything inside.		

Octal Notation

Each rwx triplet is really a 3-bit binary number in disguise

r
value 4
binary 100

w
value 2
binary 010

x
value 1
binary 001

Add the values of the permissions you want to grant — each triplet's total is one digit, 0 to 7

rwx	Calculation	Meaning
rw x	4+2+1 = 7	read, write, execute
rw-	4+2+0 = 6	read, write
r-x	4+0+1 = 5	read, execute
r--	4+0+0 = 4	read only
---	0+0+0 = 0	no permission

```
755 = rwx r-x r-x    (owner: rwx, group: r-x, others: r-x)
```

chmod — Symbolic Mode

Change permissions using letters: who, action, and which permission

```
$ chmod u+x filename
```

WHO

u (owner) g (group) o (others) a
(all)

ACTION

+ add - remove = set
exactly

PERMISSION

r w x

Examples

```
chmod u+x script.sh
```

Give the owner execute permission

```
chmod g-w notes.txt
```

Remove write permission from the group

```
chmod o=r file.txt
```

Set others' permission to exactly read-only

```
chmod a+r shared.txt
```

Give everyone read permission

```
chmod u+rwx,g+rx,o-rwx file
```

Set different permissions for each identity in one command

chmod — Numeric (Octal) Mode

Set all three identities at once using a 3-digit octal number

```
$ chmod 755 script.sh
```

7

Owner · rwx

5

Group · r-x

5

Others · r-x

Common permission sets to know

644	rw-r--r--	Standard file: owner edits, everyone reads
755	rxrx-r-xr-x	Standard script/program: owner edits+runs, others run
600	rw-----	Private file: only owner can read/write
777	rxrxrxrxrx	Everyone can do everything — avoid unless truly needed

chown and chgrp — Changing Ownership

Permissions decide what's allowed; ownership decides who those rules apply to

chown

Changes the owner (and optionally the group) of a file

```
$ chown alice file.txt
```

```
$ chown alice:staff file.txt
```

```
$ chown -R alice project/
```

-R apply recursively to a whole directory tree. Requires **sudo** unless you already own the file.

chgrp

Changes only the group associated with a file

```
$ chgrp students notes.txt
```

```
$ chgrp -R students project/
```

Tip: `chown user:group file` is a shortcut that changes both owner and group in one command — same result as `chown` + `chgrp` separately.

umask — Default Permission Mask

Controls what permissions brand-new files and directories get automatically

```
$ umask
```

```
0022
```

- umask is subtracted from the system's maximum default (666 for files, 777 for directories)
- Files never get execute permission by default, even if umask allows it — that has to be added explicitly with chmod
- A higher umask value means fewer default permissions granted

Worked example — umask 022

Item	Default max	umask	Result
New file	666 (rw-rw-rw-)	- 022	644 (rw-r--r--)
New directory	777 (rwxrwxrwx)	- 022	755 (rwxr-xr-x)

Key Takeaways

1

`rwX / owner-group-others`

the entire permission model: 3 permissions x 3 identities

2

`Octal (e.g. 755)`

`r=4, w=2, x=1` — add them up per identity, one digit each

3

`chmod`

changes permissions — symbolic (`u+x`) or numeric (`755`)

4

`chown / chgrp`

change who owns a file, and which group it belongs to

5

`umask`

sets the default permissions new files and folders start with